Les collections en Python

- Tuples
- Sets
- Dictionnaires

Coordinatrice : Cherifa BEN KHELIL
cherifa.ben-khelil@efrei.fr
Support du cours: Asma Gabis

P1, P1-BN, P1-P, P1-BDX – 2024/2025

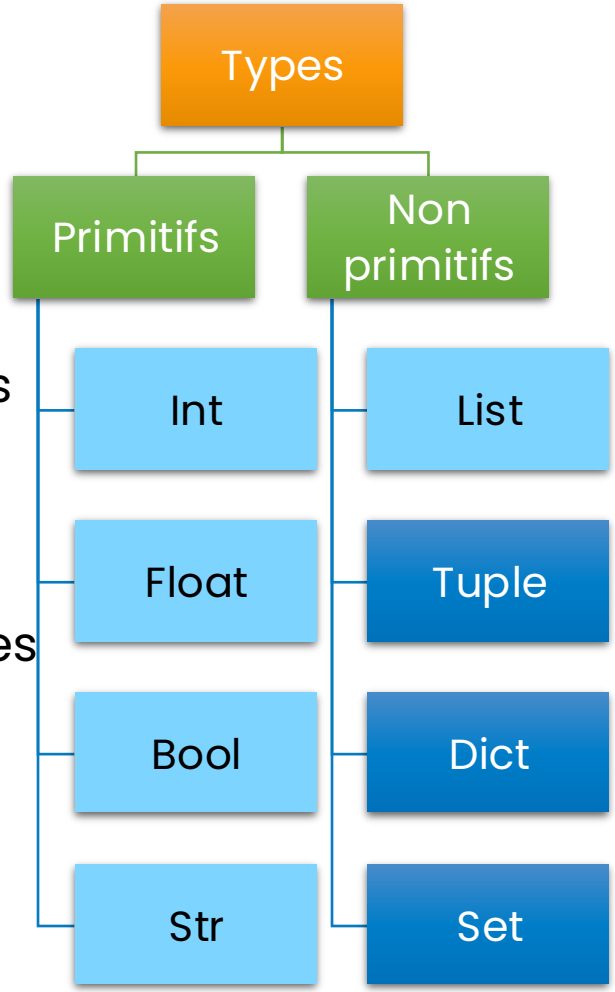
Les collections en Python

Types de données

En fonction du nombre d'éléments stockés dans une variable, nous pouvons distinguer deux catégories de types:

- **Les types primitifs (ou de base)** : **int**, **float**, **str**, **bool** déjà manipulés depuis le début de ce cours.

- **Les types non primitifs (ou structurés)** : construits à partir des types de base. Ils représentent une collection d'éléments.
  - Le type **list** a déjà été abordé dans ce cours.



Les collections en Python

Pourquoi plusieurs types non-primitifs ?

Tous les types non-primitifs représentent une collection d'éléments de types différents.

MAIS

Chaque type non-primitif a :

- Sa règle pour indexer ses éléments
- Sa règle pour parcourir ses éléments
- Sa règle pour ordonner ses éléments
- Sa règle pour modifier ses éléments

01

Les tuples

01

Les tuples

Définition

- Un tuple se crée en utilisant des parenthèses **()**
- Un tuple est une structure de données **hétérogène** : il peut accueillir des éléments de types différents.
- Un tuple est **immuable** : Une fois créé, un tuple ne peut pas être modifié.
- Un tuple est une séquence **ordonnée** : les éléments figurent dans l'ordre où ils ont été insérés.

```python
# Création d'un tuple vide
t1 = ()
print(t1)
# Création d'un tuple avec des éléments de différents types primitifs
t2 = (1, 4.5, 'a', 'abc')
print(t2)

# Création d'un tuple avec des éléments de types divers
t3 = (1, 4.5, "a", [12, 56], ("abc", 12.5))
print(t3)
```

```
()
(1, 4.5, 'a', 'abc')
(1, 4.5, 'a', [12, 56], ('abc', 12.5))
```

Les tuples

Accéder aux éléments d'un tuple

- Il est possible d'accéder aux éléments d'un tuple en utilisant des indices entre **crochets**.
- Les indices sont numérotés à partir de **0**.
  - Lorsque l'élément cible **est l'un des items du tuple lui-même:** utiliser un seul indice représentant la position de l'élément dans le tuple.

```
# Tuple avec des items de types primitifs
t2 = (1, 4.5, 'a', 'abc')
print(t2[0])
print(t2[2])
print(t2[3])
# Tuple avec des items de types collection
t3 = (1, 4.5, "a", [12, 56], ("abc", 12.5))
print(t3[3])
```

```
1
a
abc
[12, 56]
```

Les tuples

Accéder aux éléments d'un tuple

- Il est possible d'accéder aux éléments d'un tuple en utilisant des indices entre **crochets**.
- Les indices sont numérotés à partir de **0**.
  - Lorsque l'élément cible **est un élément de l'un des items de type collection** :
    - Utiliser le premier indice pour accéder à la collection (sa position) portant l'élément cible dans le tuple
    - Puis, ajouter autant d'indices que nécessaire pour atteindre l'élément cible.

```
# Tuple avec des items de types collection
t3 = (1, 4.5, "a", [12, 56], ("abc", 12.5), [[1,2], [3,4]])
print(t3[3][1])
print(t3[4][0])
print(t3[5][1][1])
```

```
56
abc
4
```

Les tuples

Créer un tuple à un seul élément

```
t = (3)

print(type(t))

print(t[0])
```

```
<class 'int'>
```

ATTENTION !!! Les parenthèses ici sont considérées comme faisant partie de l'expression mathématique (comme pour écrire **t = (3+2)**)

```
TypeError: 'int' object is not subscriptable
```

Ajouter une virgule pour signifier que **t** est un tuple

Solution:

```
t = (3,)
print(type(t))
print(t)
print(t[0])
```

```
<class 'tuple'>
(3,)
3
```

Les tuples

Tuple Vs List : Différence

- Le type tuple peut être confondu avec le type List

MAIS

- Le type List est **mutable** : les éléments de la liste peuvent être modifiés après initialisation.

```
alphagrec = ["alpha", "beta"]
print(alphagrec)
alphagrec[1] = "gamma"
print(alphagrec)
```

```
['alpha', 'beta']
['alpha', 'gamma']
```

- Le type Tuple est **immuable** : les éléments du tuple ne peuvent être modifiés après initialisation.

```
alphagrec = ("alpha", "beta")
print(alphagrec)
alphagrec[1] = "gamma"
print(alphagrec)
```

```
    alphagrec[1] = "gamma"
TypeError: 'tuple' object does not support item assignment
```

Les tuples

Tuple Vs List : Similarités

- Le type tuple présente plusieurs points communs avec le type List

Fonctionnalités appliquées sur le type List et valides sur le type **Tuple**	Exemple avec le type **Tuple**

Indexage négatif: Comme pour le type List, les indices négatifs sont supportés par les Tuples.	`x = (1, 2, 3)`
	`print(x[-1])`
	3

Slicing : Les mêmes règles appliquées sur le type List sont valides pour Tuple.	`x = (1, 2, 3)`
	`print(x[:2])`
	(1, 2)

Longueur : La fonction **len** est applicable sur un tuple pour donner sa longueur.	`x = (1, 2, 3, 2)`
	`print(len(x))`
	4

Les tuples

Tuple Vs List : Similarités

- Le type tuple présente plusieurs points communs avec le type List

Fonctionnalités appliquées sur le type List et valides sur le type **Tuple**	Exemple avec le type **Tuple**

Parcours par index	`x = (1, 2, 3, 2)`
	`for i in range(len(x)):`
	`  print(x[i], end=" ")`
	
	`1 2 3 2`

Parcours par itérateur	`x = (1, 2, 3, 2)`
	`for val in x:`
	`  print(val, end=" ")`
	
	`1 2 3 2`

Concaténation	`x1 = (1, 2)`
	`x2 = (3, 4)`
	`print(x1 + x2 + (5,))`
	
	`(1, 2, 3, 4, 5)`

Les tuples

Affectations multiples

- En Python, il est possible de faire une affectation multiple grâce aux tuples.

```
# Avec ou sans parenthèses, c'est pareil
(a, b) = (3, 2)
print("a = {} et b = {}".format(a, b))

# Les parenthèses ne sont pas obligatoires
a, b = b, a # permutation de a et b
print("a = {} et b = {}".format(a, b))
```

```
a = 3 et b = 2
a = 2 et b = 3
```

Les tuples

Un tuple retourné par une fonction

Comment appeler une fonction retournant un tuple (plusieurs valeurs) ?

```
# fonction qui demande de saisir le nom et l'age
def nom_age():
    nom = input("Saisir votre nom : ")
    age = input("Saisir votre age : ")
    return nom, age # <-- c'est un tuple
```

```
# 2 façons d'appeler la fonction
# Cas 1
n, a = nom_age()
print("Votre nom est {} et votre âge est de {} ans.".format(n, a))
```

```
Saisir votre nom : Emilie
Saisir votre age : 19
Votre nom est Emilie et votre âge est de 19 ans.
```

```
# Cas 2 : utilisation des indices
info  = nom_age()
print("Votre nom est {} et votre âge est de {} ans.".format(info[0], info[1]))
```

```
Saisir votre nom : Jean
Saisir votre age : 25
Votre nom est Jean et votre âge est de 25 ans.
```

Les tuples

Conversions avec les tuples

Convertir un tuple en liste : Il suffit de caster par le type List

```
mon_nom_t = ("Olivier", "Dupont")
print(type(mon_nom_t))
mon_nom_l = list(mon_nom_t)
print(type(mon_nom_l))
print(mon_nom_l)
mon_nom_l[0] = "Françoise"
print(mon_nom_l)
```

```
<class 'tuple'>
<class 'list'>
['Olivier', 'Dupont']
['Françoise', 'Dupont']
```

Convertir une liste en tuple : Il suffit de caster par le type Tuple

```
notes = [18.1, 10.9]
print(type(notes))
notes_t = tuple(notes)
print(type(notes_t))
print(notes_t)
notes_t[0] = 19.0 # tentative de modification
print(notes_t)
```

```
    notes_t[0] = 19.0 # tentative de modification
TypeError: 'tuple' object does not support item assignment
<class 'tuple'>
(18.1, 10.9)
```

Les tuples

Conversions avec les tuples

Convertir un tuple en Str : Utiliser la méthode **string.join()**. Elle prend en paramètre un tuple (ou une liste) et convertit ses éléments en une chaîne de caractères séparés par **string**.

```
# Exemple 1
lettres = ('H', 'e', 'l', 'l', 'o')
# Pas de séparateur
mot = "".join(lettres)
print(mot)
# Exemple 2
mes_cours = ("mathématique", "informatique", "physique")
# Le séparateur ici est " | "
mes_cours_str = " | ".join(mes_cours)
print(mes_cours_str)
```

```
Hello
mathématique | informatique | physique
```

Les tuples

Conversions avec les tuples

Convertir un Str en Tuple : Utiliser la méthode **string.split(separateur)**. Elle s'applique sur la chaine de caractères **string** et retourne par défaut une liste dont les éléments sont séparés par **separateur**.

```
mot = "1,-17,3,25,14"
# La méthode split le convertit par défaut en list
lettres = mot.split(",")
print(type(lettres))
print(lettres)
# Caster la liste pour la convertir en tuple
lettres_t = tuple(lettres)
print(lettres_t)
```

```
<class 'list'>
['1', '-17', '3', '25', '14']
('1', '-17', '3', '25', '14')
```

Les tuples

À l'utilisation : tuple ou liste ?

J'utilise une liste lorsque :	J'utilise un tuple lorsque :
• J'ai le même type de données • Traitement séquentiel : concept du « prochain item »	• J'ai des données de types hétérogènes • Traitement simultané : affectation multiple par ex • J'ai une fonction qui retourne plusieurs sorties

02

Les sets

02

Les sets

Définition

- Un set (un ensemble) est une collection de données qui se crée en utilisant les accolades **{}**
- Le set est une collection de données **non ordonnées**
- Le set est une collection de données qui n'**accepte pas les doublons** : Tous les éléments sont uniques

```
# Création et initialisation d'un set
s = {"banane", "pomme", "abricot"}

# Création d'un set vide
s = set() #<-- utilisation de set()
```

Les sets

À quoi sert le Set ?

1. Pour éliminer les doublons dans une collection de données

```
# Définition d'un set avec des doublons
fruits = {'apple', 'orange', 'apple', 'pear', 'orange', 'banana'}
# Affichage du set
print(fruits)
```

```
{'pear', 'apple', 'banana', 'orange'}
```

Les doublons ont été éliminés automatiquement

2. Pour tester l'appartenance d'un élément à une collection rapidement

```
# Définition d'un set avec des doublons
fruits = {'apple', 'orange', 'apple', 'pear', 'orange', 'banana'}
# Affichage du résultat du test d'appartenance
print("apple" in fruits )

# Affichage du résultat du test d'appartenance
print("grapes" in fruits )
```

```
True
False
```

Les sets

À quoi sert le Set ?

3. Pour faire l'union : utilisation de l'opérateur |

```
# Pour supprimer les lettres redondantes dans a
a = set('abracadabra')
# Pour supprimer les lettres redondantes dans b
b = set('alacazam')
# Lettres se trouvant dans a ou b ou dans les deux
print(a | b)
```

```
{'b', 'r', 'd', 'm', 'z', 'l', 'a', 'c'}
```

4. Pour faire l'intersection : utilisation de l'opérateur &

```
# Pour supprimer les lettres redondantes dans a
a = set('abracadabra')
# Pour supprimer les lettres redondantes dans b
b = set('alacazam')
# Lettres se trouvant à la fois dans a et dans b
print(a & b)
```

```
{'a', 'c'}
```

Les sets

À quoi sert le Set ?

5. La différence : utilisation de l'opérateur -

```
# Pour supprimer les lettres redondantes dans a
a = set('abracadabra')
# Pour supprimer les lettres redondantes dans b
b = set('alacazam')
# Lettres se trouvant dans a mais pas dans b
print(a - b)
```

```
{'d', 'b', 'r'}
```

Les sets

Opérations sur un Set

- L'ajout d'un élément se fait par la fonction **add()**

```
majuscules = {'A', 'B', 'C'}
print(majuscules)
majuscules.add('D')
# La lettre D srea bien ajoutée
print(majuscules)
majuscules.add('C')
# L'ajout de la lettre C sera sans effet car existe déjà dans le set
print(majuscules)
```

```
{'B', 'C', 'A'}
{'D', 'B', 'C', 'A'}
{'D', 'B', 'C', 'A'}
```

L'ordre d'affichage est aléatoire

Les sets

Opérations sur un Set

- **len()** donne le nombre d'éléments dans un set

```
majuscules = {'A', 'B', 'C'}
print(len(majuscules))
majuscules.add('D')
# La lettre D srea bien ajoutée
print(len(majuscules))
majuscules.add('C')
# L'ajout de la lettre C sera sans effet la taille ne changera pas
print(len(majuscules))
```

```
3
4
4
```

- Supprimer un élément d'un set par la fonction **remove()**

```
majuscules = {'A', 'B', 'C'}
print(len(majuscules))
# Suppression d'un élément du set
majuscules.remove('C')
print(majuscules)
```

```
{'B', 'A'}
```

Les sets

Parcourir un Set

```
Nombres = {2, 22, 5, 9, 11}
for val in Nombres:
    print(val)
```

```
2
5
22
9
11
```

03

Les dictionnaires

03

Les dictionnaires

Définition

- Un dictionnaire est une autre forme de structures de données qui se crée en utilisant les accolades **{}**
- La différence avec le set est que les éléments d'un dictionnaire sont composés de paires : **clef : valeur**
- Les éléments d'un dictionnaire **ne sont pas ordonnés** ➔ Pas de notion d'indice pour accéder aux valeurs.
- Dans un même dictionnaire, **les clés sont uniques** : il n'est pas possible d'avoir deux couples avec la même clé
- Les valeurs dans un dictionnaires **ne sont pas** forcément **uniques**.

```
jours = {1: "lundi", 2:"mardi", 3:"mercredi", 4:"jeudi", 5:"vendredi", 6:"samedi", 7:"dimanche"}
```

: clés
: Valeurs

Les dictionnaires

Comment accéder aux valeurs un dictionnaire ?

- Il n'est pas possible d'utiliser des indices
- L'accès à une valeur se fait par sa **clé**

```
jours = {1: "lundi", 2:"mardi", 3:"mercredi", 4:"jeudi", 5:"vendredi", 6:"samedi", 7:"dimanche"}
print(jours)
jours[1] = "Monday" # <-- modifie la valeur ayant pour clé 1
print(jours)
print(jours[4]) # <-- affiche la valeur ayant pour clé 4
```

```
{1: 'lundi', 2: 'mardi', 3: 'mercredi', 4: 'jeudi', 5: 'vendredi', 6: 'samedi', 7: 'dimanche'}
{1: 'Monday', 2: 'mardi', 3: 'mercredi', 4: 'jeudi', 5: 'vendredi', 6: 'samedi', 7: 'dimanche'}
jeudi
```

Les dictionnaires

Créer et remplir un dictionnaire

```
mois = {}
# Création d'une paire 1:"janvier"
mois[1] = "janvier"
# Création d'une paire 9:"septembre"
mois[9] = "septembre"
print(mois)
```

```
{1: 'janvier', 9: 'septembre'}
```

Ici 9 est considérée comme clé de la valeur « septembre »

Supprimer un élément du dictionnaire

Utilisation de la commande **del nom_dic** [**clé**]

```
# Création d'un dictionnaire vide
mois = {}
# Création d'une paire 1:"janvier"
mois[1] = "janvier"
# Création d'une paire 9:"septembre"
mois[9] = "septembre"
print(mois)
# Suppression de la paire dont la clé est 9
del mois[9]
print(mois)
```

```
{1: 'janvier', 9: 'septembre'}
{1: 'janvier'}
```

Les dictionnaires

Qu'y a t-il dans un dictionnaire ?

- **Items** (les paires clé:valeur) : accessibles via la méthode **items()** en utilisant la syntaxe **: nom_dict.items()**

```
jours = {1: "lundi", 2:"mardi", 3:"mercredi", 4:"jeudi", 5:"vendredi", 6:"samedi", 7:"dimanche"}
print(jours.items())
```

```
dict_items([(1, 'lundi'), (2, 'mardi'), (3, 'mercredi'), (4, 'jeudi'), (5, 'vendredi'), (6, 'samedi'), (7, 'dimanche')])
```

Les dictionnaires

Qu'y a t-il dans un dictionnaire ?

- **Clés** : accessibles via la méthode **keys()** en utilisant la syntaxe : **nom_dict**.**keys()**

```
jours = {1: "lundi", 2:"mardi", 3:"mercredi", 4:"jeudi", 5:"vendredi", 6:"samedi", 7:"dimanche"}
print(jours.keys())
```

```
dict_keys([1, 2, 3, 4, 5, 6, 7])
```

- **Valeurs** : accessibles via la méthode **values()** en utilisant la syntaxe : **nom_dict**.**values()**

```
jours = {1: "lundi", 2:"mardi", 3:"mercredi", 4:"jeudi", 5:"vendredi", 6:"samedi", 7:"dimanche"}
print(jours.values())
```

```
dict_values(['lundi', 'mardi', 'mercredi', 'jeudi', 'vendredi', 'samedi', 'dimanche'])
```

Les dictionnaires

Parcourir un dictionnaire

- **Par items :**

```
jours = {1: "lundi", 2:"mardi", 3:"mercredi", 4:"jeudi", 5:"vendredi", 6:"samedi",
7:"dimanche"}
for jour in jours.items():
    # affichage des items sous forme de tuples
    print(jour)
```

```
(1, 'lundi')
(2, 'mardi')
(3, 'mercredi')
(4, 'jeudi')
(5, 'vendredi')
(6, 'samedi')
(7, 'dimanche')
```

```
clé = 1 , valeur = lundi
clé = 2 , valeur = mardi
clé = 3 , valeur = mercredi
clé = 4 , valeur = jeudi
clé = 5 , valeur = vendredi
clé = 6 , valeur = samedi
clé = 7 , valeur = dimanche
```

```
jours = {1: "lundi", 2:"mardi", 3:"mercredi", 4:"jeudi", 5:"vendredi", 6:"samedi",
7:"dimanche"}
# Utiliser un tuple comme itérateur
for (a, b) in jours.items():
    # affetation multiple de la clé et de la valeur à chaque itération
    print("clé = {} , valeur = {}".format(a, b))
```

Les dictionnaires

Parcourir un dictionnaire

- **Par clés :**

```
jours = {1: "lundi", 2:"mardi", 3:"mercredi", 4:"jeudi", 5:"vendredi", 6:"samedi",
7:"dimanche"}
for cle in jours.keys():
    # affichage des clés
    print(cle)
```

```
1
2
3
4
5
6
7
```

```
lundi
mardi
mercredi
jeudi
vendredi
samedi
dimanche
```

```
jours = {1: "lundi", 2:"mardi", 3:"mercredi", 4:"jeudi", 5:"vendredi", 6:"samedi",
7:"dimanche"}
for cle in jours.keys():
    # affichage des valeurs correspondantes aux clés
    print(jours[cle])
```

Les dictionnaires

Parcourir un dictionnaire

- **Par valeurs :**

```
jours = {1: "lundi", 2:"mardi", 3:"mercredi", 4:"jeudi", 5:"vendredi", 6:"samedi", 7:"dimanche"}
for val in jours.values():
    # affichage des valeurs
    print(val)
```

```
lundi
mardi
mercredi
jeudi
vendredi
samedi
dimanche
```

04

Activité

04

Activité

Exercice 1

Étant donné une liste de chaînes de caractères, créer un dictionnaire où chaque paire représente un mot associé au nombre d'occurrence de ce mot dans la liste.

- **Input:** `["the", "cat", "sat", "on", "the", "mat"]`
- **Output :** `{"the":2, "cat":1, "sat":1, "on":1, "mat":1}`

Exercice 2

Écrire une fonction qui prend en paramètres un dictionnaire ayant des clés de type entier et qui retourne l'item (paire **clé : valeur**) ayant la plus petite clé.